

DP Computer Science: B 1.1.3 Applying Computational Thinking

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Explain how the fundamental concepts of computational thinking (**abstraction, algorithmic design, decomposition, pattern recognition**) are applied in various areas of computer science
 - Describe how computational thinking can be used as a **problem-solving toolkit** independently of programming
 - Provide **real-world examples** of how computational thinking is used in:
 - **Software development**
 - **Data analysis**
 - **Machine learning**
 - **Database design**
 - **Network security**
 - **Analyse** a given computer science problem and explain how computational thinking principles can be used to solve it
 - **Articulate** the relationship between computational thinking and problem-solving in computer science
-

Key Vocabulary

Term	Definition
Computational thinking	A problem-solving approach using decomposition, pattern recognition, abstraction, and algorithmic design
Software development	The process of designing, creating, and maintaining software
Data analysis	The process of inspecting, cleaning, and modelling data to discover useful information
Machine learning	A subset of AI that enables systems to learn from data without being explicitly programmed
Database design	The process of structuring a database to store and manage data efficiently
Network security	The protection of network infrastructure from unauthorised access or attacks

ATL Skills

Skill	Description
Thinking (Critical Thinking)	Analysing complex problems and identifying how CT strategies can be applied
Thinking (Creative Thinking)	Generating solutions using CT principles
Transfer Skills	Applying CT concepts to diverse areas within computer science and beyond

Skill	Description
Communication	Clearly explaining the application of CT in various contexts

2. CT in Computer Science: The Big Picture

How CT Applies Across Computer Science



CT as a Toolkit (Not Just Programming)

"Computational thinking is a mindset—not just a coding skill. It can be applied to any problem, whether or not code is written."

3. CT in Software Development

CT Concept	Application in Software Development
Decomposition	Breaking a large software project into smaller modules (e.g., user interface, database, business logic)
Pattern Recognition	Identifying reusable code patterns (e.g., design patterns, MVC architecture) to improve efficiency
Abstraction	Creating simplified interfaces (e.g., APIs) for complex systems; using frameworks to hide complexity

CT Concept	Application in Software Development
Algorithmic Design	Developing algorithms for specific tasks within the software (e.g., sorting, searching, authentication)

Example: Building a Web Application



4. CT in Data Analysis

CT Concept	Application in Data Analysis
Decomposition	Organising large datasets into manageable subsets
Pattern Recognition	Identifying trends, anomalies, and correlations in data
Abstraction	Creating visualisations (charts, graphs) to represent complex data
Algorithmic Design	Developing algorithms for data processing and analysis (e.g., cleaning, sorting, filtering)

Example: Website Traffic Analysis

Step	CT Concept	Application
1	Decomposition	Group data by date, page visits, user location
2	Pattern Recognition	Identify most popular pages, peak traffic times

Step	CT Concept	Application
3	Abstraction	Create bar charts and graphs to visualise findings
4	Algorithmic Design	Create rules to identify anomalies (e.g., if traffic > 1000 per hour, flag for review)

5. CT in Machine Learning

CT Concept	Application in Machine Learning
Decomposition	Breaking the learning process into steps: data collection, preprocessing, training, evaluation
Pattern Recognition	Algorithms identifying patterns in data to make predictions
Abstraction	Creating simplified models that represent real-world phenomena
Algorithmic Design	Developing algorithms for training and applying machine learning models

Example: Recommendation System

Step	CT Concept	Application
1	Decomposition	Break down: data collection, analysis, filtering, ranking
2	Pattern Recognition	Identify patterns in user behaviour and content preferences
3	Abstraction	Create a simplified model of user interests and content relevance
4	Algorithmic Design	Create step-by-step algorithm for generating recommendations

6. CT in Database Design

CT Concept	Application in Database Design
Decomposition	Organising data into tables and relationships
Pattern Recognition	Identifying data dependencies and redundancies (normalisation)
Abstraction	Creating schemas to represent the database structure (conceptual, logical, physical)
Algorithmic Design	Developing algorithms for data retrieval and manipulation (SQL queries)

Example: Library Database

Step	CT Concept	Application
1	Decomposition	Create tables: Books, Members, Loans, Categories
2	Pattern Recognition	Identify relationships: Member borrows Book (1:N), Book belongs to Category (1:N)
3	Abstraction	Create ERD to represent the database structure
4	Algorithmic Design	Write SQL queries for data retrieval (e.g., SELECT, JOIN)

7. CT in Network Security

CT Concept	Application in Network Security
Decomposition	Analysing network components and potential vulnerabilities
Pattern Recognition	Identifying attack patterns and anomalies in network traffic
Abstraction	Creating models to simulate network behaviour and security threats
Algorithmic Design	Developing algorithms for intrusion detection and prevention

Example: Intrusion Detection System

Step	CT Concept	Application
1	Decomposition	Break down network into components (routers, servers, clients)
2	Pattern Recognition	Identify unusual traffic patterns (e.g., DDoS attacks)
3	Abstraction	Create a model of normal network behaviour
4	Algorithmic Design	Create algorithm to detect and respond to security threats

8. Activity: The "Data Detective"

Scenario

You are managing the social media account for your school's "Code Crusaders" computer science club. You want to increase student engagement. Your goal is to analyse the performance of the last 15 posts to figure out what kind of content works best and when you should post it.

Dataset

Post ID	Post Type	Day of Week	Time of Day	Likes	Shares
1	Image	Monday	Afternoon	25	5
2	Link	Monday	Evening	12	2

Post ID	Post Type	Day of Week	Time of Day	Likes	Shares
3	Video	Tuesday	Evening	95	35
4	Text	Wednesday	Morning	8	1
5	Image	Wednesday	Evening	45	12
6	Link	Thursday	Afternoon	15	3
7	Video	Friday	Evening	110	42
8	Image	Saturday	Afternoon	60	25
9	Text	Saturday	Morning	10	2
10	Link	Sunday	Morning	9	1
11	Image	Sunday	Evening	55	22
12	Video	Monday	Evening	88	31
13	Link	Tuesday	Morning	11	2
14	Image	Thursday	Evening	48	15
15	Text	Friday	Afternoon	18	4

Applying CT to the Data

CT Concept	Application
Decomposition	Group posts by Post Type, Time of Day, and Day of Week
Pattern Recognition	Identify that videos have the highest likes (avg 98); evening posts perform best; weekend posts have higher engagement
Abstraction	Create a bar chart showing average likes per post type
Algorithmic Design	Create rules for optimal posting strategy: "If goal is to maximise engagement, create a Video post, publish in the Evening, and prioritise Friday or the weekend."

Expected Findings

Observation	Insight
Post Type	Videos are most popular (avg 98 likes, 36 shares)
Time of Day	Evening posts perform better (higher engagement)
Day of Week	Weekend posts (Saturday, Sunday) get more engagement, especially shares
Anomaly	Post 7 (Video on Friday evening) was extremely successful (110 likes)

9. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	10 min	Recap 4 CT concepts; pose the question: "How do these concepts help us solve problems in computer science?"
CT in CS	20 min	Teacher-led discussion: Software development, Data analysis, Machine learning, Database design, Network security
Activity	25 min	"Data Detective" — groups analyse a dataset using CT
Presentation	10 min	Groups present findings; peer feedback
Conclusion	5 min	CT beyond programming; recap and summary

10. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Group Activity	Assess ability to apply CT principles to data analysis
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Questions:

- 1. How can decomposition be applied to software development?
- 2. What is the role of pattern recognition in data analysis?
- 3. Give an example of abstraction in machine learning.
- 4. Why is algorithmic design important in network security?

11. Differentiation

Level	Strategy
Support	Provide scaffolded examples; use relatable scenarios
Challenge	Explore advanced applications of CT in AI, cybersecurity, and big data

12. Resources

- Presentation (Slide Deck)
- eBook (B1.1.3 with examples)
- Quizlet (B1.1.3 vocabulary flashcards)
- Activity: Data Detective scenario

13. Summary: Key Takeaways

Area	CT Application
Software Development	Decomposition (modules), Pattern Recognition (design patterns), Abstraction (APIs), Algorithmic Design (algorithms)
Data Analysis	Decomposition (subsets), Pattern Recognition (trends), Abstraction (visualisations), Algorithmic Design (algorithms)
Machine Learning	Decomposition (steps), Pattern Recognition (algorithms), Abstraction (models), Algorithmic Design (training)
Database Design	Decomposition (tables), Pattern Recognition (dependencies), Abstraction (schemas), Algorithmic Design (SQL)
Network Security	Decomposition (components), Pattern Recognition (attacks), Abstraction (models), Algorithmic Design (detection)

text

KEY REMINDERS
✔ CT is a problem-solving toolkit for all of CS ✔ Decomposition = break down large problems ✔ Pattern Recognition = find trends and similarities ✔ Abstraction = focus on essentials ✔ Algorithmic Design = create step-by-step solutions ✔ CT can be applied WITHOUT coding

"Computational thinking is the foundation of modern computer science—it's how we solve complex problems across every domain."